

Architecture of Mathematics: Internal Languages, Categorical Models, and the Spectral Lattice of Formal Systems

Groupoid Infinity

May 30, 2026

Abstract

We analyze the Groupoid Infinity stack of formal languages as a *lattice* of internal languages, not a linear tower. The languages fall into three distinct branches: a *constructive type theory and set analysis* spine (Henk–Frank–Christine), an Homotopy Type Theory and Super Differential Smooth Quantum Spectra *super homotopy* spine (Henk–Per–Anders–Urs), and a *infinity category theory* spine (Dan–Mike–Ulrik). These spines are connected by functorial relationships but their syntactic term structures are fundamentally incompatible and should not be unified into a single AST. In particular, simplicial type theory (Ulrik/STT) is connected to Urs via directed-homotopy foundations, providing the simplicial backbone for its 3-layer architecture: Super-Smooth, Differential K-Theory, and Quantum Linear types. We construct syntactic CW-complexes and chain complexes for each language individually, define their cohomology groups, and assemble a spectral sequence over the full lattice of language categories.

Contents

1	Categorical Foundations	3
2	The Three Spines of the Language Lattice	4
2.1	The Topoi Structure	5
2.2	The Semantic Foundations: Structural, Cohesive, and Directed Modalities	6
3	Spine I: Set Analysis	8
3.1	Frank: Minimal CIC	8
3.2	Christine: Full CIC (CwF/CwA)	8
3.3	Laurent: Calculus and Functional Analysis	8
4	Spine II: Super Homotopy Type Theory	9
4.1	Henk: Pure Type Systems (PTS)	9
4.2	Per: Martin-Löf Type Theory (MLTT-80)	9
4.3	Anders: Cubical HoTT	10
4.4	Urs: The Family of Layered Sublanguages	10
4.4.1	Felix (Layer I)	11
4.4.2	Jack (Layer II)	11
4.4.3	Urs (Layer III)	11
5	Spine III: Infinity Category Theory	12
5.1	Dan: Operational Algebra	12
5.2	Mike: Directed 1-Category Type Theory	12
5.3	Ulrik: Simplicial ∞ -Categories	13
6	Cohomology of Programming Languages	14
6.1	Physical Interpretations of Cohomology Groups	15
6.2	Examples across the Spines	16
7	Spectral Sequence of Language Categories	17
7.1	Functorial Conversions and Cross-Branch Morphisms	18
7.1.1	Deep Dive: The Lift Functor (Dan $\rightarrow$ Ulrik)	19
7.1.2	Deep Dive: The Groupoid Core (Ulrik $\rightarrow$ Anders)	19
7.1.3	Case Study: Möbius Strip Representation	20

1 Categorical Foundations

Definition 1.1 (Categories with Families (CwF)). A CwF consists of a category $\mathcal{C}$ of contexts and substitutions with a terminal object, equipped with a functor $Ty : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$, a presheaf of terms $Tm : (\int Ty)^{\text{op}} \rightarrow \mathbf{Set}$, and a context comprehension operation with projection morphisms.

Definition 1.2 (Natural Models). Following Awodey, a natural model is a category $\mathcal{C}$ with a terminal object $\mathbf{1}$ and a representable natural transformation $p : Tm \rightarrow Ty$ between presheaves. Pullbacks of p along Yoneda provide context extension and substitution.

Definition 1.3 (Grothendieck Fibration). A functor $p : \mathcal{E} \rightarrow \mathcal{B}$ is a fibration when for every $X \in \mathcal{E}$ and $u : J \rightarrow p(X)$ in $\mathcal{B}$ there exists a Cartesian lift $f : Y \rightarrow X$ in $\mathcal{E}$.

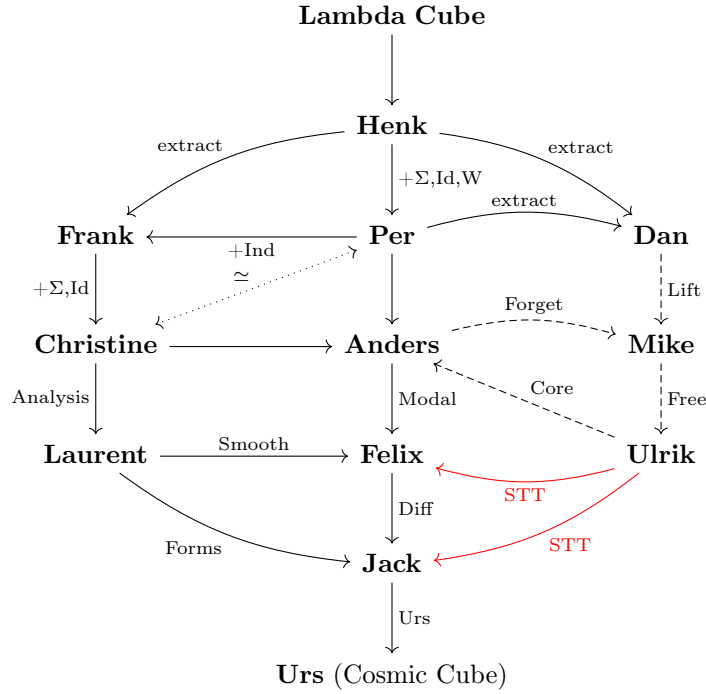
The category of languages $\mathcal{L}$ has formal languages as objects and syntactic functors (compilers, type checkers, normalizers, extractors) as morphisms.

Table 1: Semantic Categories and Core Language Primitives

Language / Primitive	Semantic Category / Model	Application & Physical/Mathematical Domain
O_λ	Cartesian Closed Category (CCC)	Untyped λ -calculus (evaluation and interpretation)
O_π	Symmetric Monoidal Category	Process calculi (CCS, CSP, Milner's π -calculus)
O_μ	Tensor Category	Tensor calculus (vectorization)
Henk (O_Π)	Π-Cat (C-systems / CwF)	PTS Calculus of Constructions (functional completeness)
Frank	Π-Cat + Ind	Minimal Calculus of Inductive Constructions (strictly positive schemes)
Christine (O_Σ)	LCCC + Ind	Full CIC (MLTT-72 contextual completeness, dependent sums)
Laurent	Top/Met	Calculus and Functional Analysis (topological and metric spaces)
Per (O_W)	LCCC + W	Martin-Löf Type Theory MLTT-80 (inductive W-types)
Anders (O_I)	∞-Cat	Cubical Type Theory with Path types, no HITs (∞ -categories)
Anders+	∞-Topos	Cubical Type Theory with HITs (lightweight core, univalence)
Felix ($O_{\dashv}$)	Cohesive ∞-Topos	Modal HoTT (synthetic differential supergeometry, flat/sharp)
Jack	Diff ∞-Topos	Stable homotopy (spectra, differential forms, K-theory)
Urs (O_H)	Urs ∞-Topos (Linear Types)	Quantum linear types (Hadamard operator, qubits, braids)
Dan ($O_{\triangleright}$)	Pres (Simplicial Sets / Chains)	Guarded recursion, operational algebra (face/degeneracy maps)
Mike	1-Cat	Directed 1-Category theory (directed paths, linear structures)
Ulrik	$(\infty, 1)$-Cat (Riehl-Shulman STT)	Simplicial $(\infty, 1)$ -categories (Segal, directed interval)

2 The Three Spines of the Language Lattice

The Groupoid Infinity stack does *not* form a linear tower. It decomposes into three distinct vertical columns: a constructive/inductive spine on the left (**Frank–Christine–Laurent**), a homotopical/cohesive spine in the center (**Henk–Per–Anders–Felix–Jack**), and an operational/directed branch on the right (**Dan–Mike–Ulrik**). The relationships between columns are functorial (lifts, free constructions, forgetful maps, smooth completions) but the syntaxes themselves are disjoint.

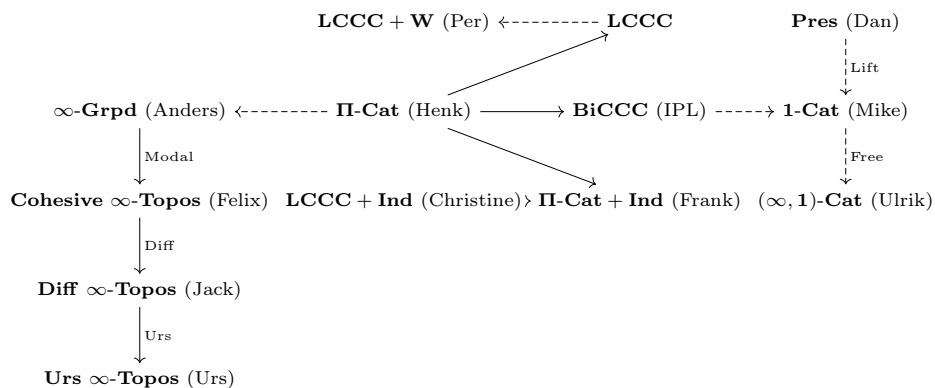


Remark 2.1 (Christine $\simeq$ Per). Christine (full CIC with Σ , Id, Ind, Prop) and Per (MLTT-80 with Σ , Id, W, 0, 1, 2) are at a comparable level of expressiveness. Both serve as the foundation for Anders and can be considered interchangeable base systems. The choice between inductive schemes (Frank/Christine path) and W-types (Per path) is a design decision, not a strict ordering.

2.1 The Topoi Structure

To analyze the gradual expansion of language primitives, we construct a fine-grained lattice of semantic categories radiating from the category of dependent products (**Π -Cat** (Henk)). The system expands in three cardinal directions (adding dependent sums to yield locally cartesian closed categories **LCCC**, inductive algebras **Π -Cat + Ind** (Frank), and bicartesian closed structures **BiCCC** (IPL)) before merging into rich categories of models such as **LCCC + Ind** (Christine), **LCCC + W** (Per), **∞ -Grpd** (Anders), and cohesive **∞ -topoi**. On the right, the lattice incorporates the operational algebra branch (**Pres** (Dan)), the directed 1-category branch (**1-Cat** (Mike)), and the simplicial **$(\infty, 1)$ -category branch $(\infty, \mathbf{1})$ -Cat** (Ulrik)). All cross-branch connections are detailed in the global map in Section 2.

Theorem 2.2 (Universal Primitive Lattice). *The formal internal languages form a connected lattice structure across three spines, linked by the modal, synthetic, and directed structural morphisms shown below:*



2.2 The Semantic Foundations: Structural, Cohesive, and Directed Modalities

The lattice stratifies into structural, cohesive, linear, and directed semantics. The foundational correspondences between semantic models and their internal syntaxes progressively build up structure:

- **Structural Core:** At the base, Locally Cartesian Closed Categories (**LCCC**) provide the semantics for **MLTT**, the pure structural type theory. Upgrading the semantics to ∞ -categories adds globular path equality, yielding Cubical Type Theory with Path types but without Higher Inductive Types (HITs). Finally, a full ∞ -topos models Cubical Type Theory with HITs, which maintains a very lightweight core and syntax.
- **Cohesive Modalities:** Cohesive Type Theory (Felix) is developed specifically to model modalities such as connectedness, compactness, and infinitesimal shapes. Differential Cohesive Type Theory acts as the internal language for Differential Cohesive ∞ -topoi, adding extra structure to ∞ -topoi just as ∞ -categories add path equality to LCCCs. These are typically modeled in the base library via undefined axioms (see `cohesivett`).
- **Linear/Tensor Modalities:** Symmetric Monoidal Categories (**SMC**) supply the semantics for resource-sensitive and directed computations. Here, Process Calculus and Interaction Nets (Lafont) act as the internal language, representing modality types with special comonadic *spawn* arrows that hide the underlying run-time implementation.
- **Directed and Simplicial Topoi:** Moving toward spatial and temporal operational algebras, presheaf categories (**Pres**) provide the semantics for Guarded Type Theory (Dan). Incorporating strict directional structures leads to **1-Cat**, acting as the semantics for Directed Type Theory (Mike). Finally, scaling to higher dimensions, $(\infty, \mathbf{1})$ -**Cat** serves as the model for Simplicial Type Theory (Ulrik / Riehl-Shulman STT), extending directed paths with a Segal-like directed interval.

This establishes a rich hierarchy of correspondences:

$$\mathbf{LCCC} \xleftarrow{\text{Internal Language}} \mathbf{MLTT}$$

$$\infty\text{-}\mathbf{Cat} \xleftarrow{\text{Internal Language}} \mathbf{Cubical} \text{ (Paths, no HITs)}$$

$$\infty\text{-}\mathbf{Topos} \xleftarrow{\text{Internal Language}} \mathbf{Cubical} \text{ (with HITs)}$$

$$\mathbf{Cohesive} \infty\text{-}\mathbf{Topos} \xleftarrow{\text{Internal Language}} \mathbf{Cohesive HoTT}$$

$$\mathbf{SMC} \xleftarrow{\text{Internal Language}} \mathbf{Process Calculus / Interaction Nets}$$

$$\mathbf{Pres} \xleftarrow{\text{Internal Language}} \mathbf{Guarded Type Theory} \text{ (Dan)}$$

$$\mathbf{1-Cat} \xleftarrow{\text{Internal Language}} \mathbf{Directed Type Theory} \text{ (Mike)}$$

$$(\infty, \mathbf{1})\text{-}\mathbf{Cat} \xleftarrow{\text{Internal Language}} \mathbf{Simplicial Type Theory} \text{ (Ulrik)}$$

3 Spine I: Set Analysis

This branch represents the constructive/inductive spine of type theories and functional analysis. It is characterized by inductive algebraic constructions and real/functional analysis foundations.

3.1 Frank: Minimal CIC

Extends Henk with strictly positive inductive schemes.

```
cosmos := Uj
var := var name
pi := Π name E E | λ name E E | E E
ind := inductive name params constrs | constr i name args | ind name E args E
```

3.2 Christine: Full CIC (CwF/CwA)

Complete Calculus of Inductive Constructions with Σ, Id, and Ind.

```
cosmos := Uj
var := var name
pi := Π name E E | λ name E E | E E
sigma := Σ name E E | (E, E) | E.1 | E.2
id := Id E | ref E | idJ E
ind := inductive name params constrs | constr i name args | ind name E args E
```

3.3 Laurent: Calculus and Functional Analysis

Laurent formalizes classical real and functional analysis, distribution theory, and measure theory. It provides first-class support for reals, complex numbers, measures, limits, and integration.

```
cosmos := Prop : U0 : U1
var := var ident | hole
forall := ∀ ident E E | λ ident E E | E E
exists := ∃ ident E E | (E, E) | E.1 | E.2
base := ℕ | ℤ | ℚ | ℝ | ℂ | ℍ | ℔ | ℚn
set := Set | SeqEq | And | Or | Complement | Intersect | Power | Closure | Cardinal
q := -/~ | Quot | LiftQ | IndQ
mu := mu | Measure | Lebesgue | Bochner
lim := Seq | Sup | Inf | Limit | Sum | Union
```


4 Spine II: Super Homotopy Type Theory

This branch represents the central path of homotopical, cubical, and cohesive type systems, transitioning from pure dependent products to higher-dimensional geometry.

4.1 Henk: Pure Type Systems (PTS)

The initial object in LCC categories. Contains only Π and universes.

```
cosmos := Uj  
var := var name  
pi :=  $\Pi$  name E E |  $\lambda$  name E E | E E
```

4.2 Per: Martin-Löf Type Theory (MLTT-80)

MLTT-80 with W-types, base types 0, 1, 2, and cubical primitives.

```
cosmos := Uj | Vk  
var := var name | hole  
pi :=  $\Pi$  name E E |  $\lambda$  name E E | E E  
sigma :=  $\Sigma$  name E E | (E, E) | E.1 | E.2 | E.name  
id := Id E | ref E | idJ E  
0 := 0 | ind0 E E E  
1 := 1 | ★ | ind1 E E E  
2 := 2 | 02 | 12 | ind2 E E E  
W := W ident E E | sup E E | indW E E
```

4.3 Anders: Cubical HoTT

CCHM cubical structure with Path, Glue, and higher inductive types (Coequ, Disc).

```

cosmos := Uj | Vk
var := var name | hole
pi := Π name E E | λ name E E | E E
sigma := Σ name E E | (E, E) | E.1 | E.2
0 := 0 | ind0 E E E
1 := 1 | ★ | ind1 E E E
2 := 2 | 02 | 12 | ind2 E E E
W := W ident E E | sup E E | indW E E
id := Id E | ref E | idJ E
path := Path E | Ei | E @ E
I := I | 0 | 1 | E ∨ E | E ∧ E | ¬E
part := Partial E E | [ (E = I) → E, ... ]
sub := inc E | ouc E | E [ I ↦ E ]
kan := transp E E | hcomp E
glue := Glue E | glue E | unglue E E
coequ := coequ E | ι2 | resp | indcoequ
disc := disc E | base | hub | spoke | inddisc

```

4.4 Urs: The Family of Layered Sublanguages

Urs is not a single monolithic type system, but rather a family of three layered sublanguages where each layer builds on top of the syntax, modalities, and constructors of the previous one:

Remark 4.1 (The 3-Layer Architecture). The sublanguages represent a progressive structure for formalizing mathematical and physical concepts:

1. **Felix**: Graded universes and tensors, group actions, and super-modality operators for geometric cohesion and differential structure—bosonic/fermionic grades, flat/sharp modalities.
2. **Jack**: Homotopical foundations via spectra and groupoids, supporting differential K-theory with differential forms, connections, and refined cohomology structures.
3. **Urs**: Configuration spaces and braids for Urs statistics, supporting Urs systems and linear types (!A) for resource-sensitive Urs programming.

Each sublanguage culminates in a system tailored to formalize superpoints ($\mathbb{R}^{m|n}$), supersymmetry, and equivariant structures.

4.4.1 Felix (Layer I)

Extends the homotopy core with graded universes, tensors, group actions, and cohesive modalities ($\flat$, $\sharp$, $\Im$) for synthetic supergeometry.

$$\begin{aligned} \text{grade} &:= \mathbf{Bose} \mid \mathbf{Fermi} \\ \text{cosmos} &:= \mathbf{U}_{j,g} \\ \text{smth} &:= \mathbf{SmthSet} \mid \mathbf{Plot} \, j \, E \, E \mid \mathbf{SupSmthSet} \\ \text{cohs} &:= \flat \, E \mid \sharp \, E \mid \Im \, E \\ \text{bose} &:= \bigcirc \, E \\ \text{tensor} &:= E \otimes E \end{aligned}$$

4.4.2 Jack (Layer II)

Extends cohesion with stable homotopy (spectra, suspensions, wedges) and differential cohomology (forms, connections, and differential K-theory).

$$\begin{aligned} \text{grp} &:= \mathbf{Grpd} \, j \mid \mathbf{Comp} \, j \, E \, E \, E \\ \text{stable} &:= \mathbf{Spectrum} \mid \mathbf{Susp} \, E \mid E \wedge E \mid \mathbf{Hom}_{Spec} \, E \, E \\ \text{ku} &:= KU_G^r \, E \, E \, E \mid \mathbf{forms} \, j \, E \mid d \, j \, E \mid \mathbf{DiffKU}_G \, E \, E \, E \, E \end{aligned}$$

4.4.3 Urs (Layer III)

The top layer, adding configuration spaces, braids, and linear resource types (!A) for Urs programming.

$$\begin{aligned} \text{qubit} &:= \mathbf{Qubit} \, E \, E \mid \mathbf{app} \, E \, E \, E \mid \mathbf{fuse} \, E \, E \, E \\ \text{linear} &:= ! \, E \\ \text{topo} &:= \mathbf{Config}_j \, E \mid \mathbf{Braid}_j \, E \end{aligned}$$

Remark 4.2 (STT $\rightarrow$ Urs: The Simplicial Connection). Simplicial type theory (Ulrik/STT) provides a direct connection to Urs. The directed interval $\mathbb{I}^\rightarrow$, extension types, and modal Π in STT supply the ∞ -categorical framework that Urs’s Layer I (Homotopy) builds upon. By isolating the category of categories $\mathbf{Cat}$ as a reflective subuniverse in simplicial type theory, STT provides the required directed homotopy structures. Concretely, STT’s directed interval becomes the path/homotopy backbone, while its modal/shape operations inform Urs’s cohesive modalities ($\flat$, $\sharp$, $\Im$) in Layer II. This is represented by the STT arrow from Ulrik to Urs in the lattice diagram, which is *not* simply a factoring through Anders.

5 Spine III: Infinity Category Theory

This branch encompasses operational algebra and directed category theories, dealing with presentations of algebraic shapes and directed categorical structures.

5.1 Dan: Operational Algebra

Dan operates on a completely different level: it manipulates algebraic presentations (groups, simplicial sets, chain complexes) operationally. Its term language has no dependent types, no universes, and no Π/Σ . Instead it works with group words, permutations, matrices, face/degeneracy maps, and boundary operators.

```

type :=  Simplex | Group | Simplicial | Chain | Category
        Monoid | Ring | Field | PermGroup | MatGroup
        FpGroup | PcGroup | Module | Set | GSet
term :=  name | E ∘ E | E-1 | Ej | e | matrix args
        E + E | E · E | perm args | cycle args
        PermGroup args | MatGroup args j k
        FpGroup names args | PcGroup names args
        dj E | sj E | ∂ E | E · E | Hom E E | E ≡ E
hypo :=  decl names type | eq name E E | map name E E
        pres name E | rel E | act name E E
        orbit E E args | stabilizer E E E

```

5.2 Mike: Directed 1-Category Type Theory

A *linear* type theory with a bidirectional type checker enforcing quadraticity (each category variable occurs exactly once covariantly and once contravariantly).

```

cat :=  var name | op C | C × C
type :=  HomC E E | E ⊗ E | ∫C name E | ∫C name E
        E  $\multimap$  E | name args
term :=  id E | idJ E | idJ,cov E | idJ,contra E
        E ⊗ E | let name ⊗ name = E in E
        coend_in name name name E | let coend_in name name = E in E
        end_in name E | let end_in name name name = E in E
        λ name E | E E | var name

```

5.3 Ulrik: Simplicial ∞ -Categories

Extends Mike's foundation with directed interval $\mathbb{I}$, shape inclusions ($\varphi \subseteq \psi$), extension types, modal Π , and twisted arrow categories (A^{tw}).

To construct a type theory for synthetic category theory, one might hope to interpret type theory in the category of categories (∞ -categories or otherwise) to ensure that types realize categories. However, the category of small categories is too poorly behaved to form a model of Martin-Löf Type Theory (MLTT). Instead, Riehl and Shulman (2017) use the category of bisimplicial sets (simplicial spaces) to model synthetic $(\infty, 1)$ -categories via Rezk spaces. Simplicial Type Theory (STT) axiomatizes this framework using a directed interval and shapes, allowing one to isolate $(\infty, 1)$ -categories as a reflective subuniverse of types within the theory.

```

cosmos := U
var := var name
pi :=  $\Pi$  name E E |  $\lambda$  name E E | E E
sigma :=  $\Sigma$  name E E | (E, E) | E.1 | E.2
id := Id E E E | ref E
idir :=  $\mathbb{I}$  | 0 | 1 | E  $\leq$  E |  $\Phi \subseteq \Psi$ 
part := Partial E E | [ (E = I)  $\rightarrow$  E, ... ]
ext := Ext E E E
modalpi :=  $\Pi$  shape E E |  $\lambda$  shape E | E E
tw := Tw E | tw_p0 E | tw_p1 E
ops := E  $\vee$  E | E  $\wedge$  E |  $\neg$ E
idJ := idJ E | idJ,cov E | idJ,contra E

```

6 Cohomology of Programming Languages

Each language's syntax is analyzed *individually* as a CW-complex.

For a formal language $\mathcal{O}_x$, we associate a chain complex of abelian groups (free $\mathbb{Z}$ -modules) $C_*(\mathcal{O}_x)$ constructed from the syntax and reduction/conversion semantics. This construction models the syntactic space as a cell complex (CW-complex) where paths and higher homotopies encode the computational rules.

Definition 6.1 (Syntactic CW-Complex). For a language $\mathcal{O}_x$ with term syntax T generated by a signature Σ , the syntactic cell complex $C_*(\mathcal{O}_x)$ is defined in low dimensions by:

- **0-cells** ($C_0(\mathcal{O}_x)$) : The free $\mathbb{Z}$ -module generated by the set of basic syntactic constructors $c \in \Sigma$ (e.g., **Var**, **Pi**, **Lam**, **App** for Henk). Any term $t \in T$ has an associated constructor-occurrence representation:

$$[t] = \sum_{c \in \Sigma} \text{count}(c, t) \cdot c \in C_0(\mathcal{O}_x)$$

where $\text{count}(c, t)$ is the number of times the constructor c appears in the syntax tree of t .

- **1-cells** ($C_1(\mathcal{O}_x)$): The free $\mathbb{Z}$ -module generated by the rewriting and reduction rules $r \in \mathcal{R}$ of the language (e.g., β -reductions, evaluation steps). Each 1-cell $r : t \rightarrow t'$ represents a directed rewrite path from a redex term t to a contractum term t' .
- **2-cells** ($C_2(\mathcal{O}_x)$): The free $\mathbb{Z}$ -module generated by equivalence relations, critical pairs, confluence diagrams, and conversion equations (e.g., η -conversions, commuting conversions). A 2-cell e represents a coherence condition or a closed loop of rewrite steps.
- **Higher cells** ($C_k(\mathcal{O}_x)$ for $k \geq 3$): Coherence relations between equations, such as Mac Lane pentagon relations, path-type univalence coherences (in Anders), or quadraticity constraints (in Mike).

Definition 6.2 (Boundary Operators). The boundary operators $\partial_k : C_k(\mathcal{O}_x) \rightarrow C_{k-1}(\mathcal{O}_x)$ are defined as follows:

1. The 1-boundary operator $\partial_1 : C_1(\mathcal{O}_x) \rightarrow C_0(\mathcal{O}_x)$ is the linear map determined by the net change in constructor counts caused by a rewrite rule $r : t \rightarrow t'$:

$$\partial_1(r) = [t'] - [t]$$

2. The 2-boundary operator $\partial_2 : C_2(\mathcal{O}_x) \rightarrow C_1(\mathcal{O}_x)$ is determined by the boundary of confluence diagrams or conversion loops. If a 2-cell e represents a confluence diagram from a term s reducing to s' via two different

paths of rewrites $p_1 = (r_{1,1}, \dots, r_{1,m})$ and $p_2 = (r_{2,1}, \dots, r_{2,n})$, the boundary is:

$$\partial_2(e) = \sum_{j=1}^m r_{1,j} - \sum_{k=1}^n r_{2,k}$$

Since both paths p_1 and p_2 start at s and end at s' , the net constructor changes must match:

$$\partial_1(p_1) = [s'] - [s] = \partial_1(p_2)$$

Thus:

$$\partial_1(\partial_2(e)) = \partial_1(p_1) - \partial_1(p_2) = 0$$

This proves that $\partial_1 \circ \partial_2 = 0$, establishing a valid chain complex.

Definition 6.3 (Syntactic Cohomology). Let $C_*(\mathcal{O}_x)$ be the syntactic chain complex of $\mathcal{O}_x$ and let G be an abelian group of coefficients. The cochain complex $C^*(\mathcal{O}_x; G) = \text{Hom}(C_*(\mathcal{O}_x), G)$ has coboundary operators $\delta^k : C^k(\mathcal{O}_x; G) \rightarrow C^{k+1}(\mathcal{O}_x; G)$ defined by $\delta^k(\phi) = \phi \circ \partial_{k+1}$. The syntactic cohomology groups are:

$$H^k(\mathcal{O}_x; G) = \ker(\delta^k) / \text{im}(\delta^{k-1})$$

6.1 Physical Interpretations of Cohomology Groups

The cohomology groups $H^k(\mathcal{O}_x; G)$ measure fundamental properties of the syntax and semantics of $\mathcal{O}_x$:

- $H^0(\mathcal{O}_x; G)$: Measures the *syntactic dimension* or independent constructor classes. A class in H^0 is a function $f : C_0 \rightarrow G$ such that for any rewrite $r : t \rightarrow t'$, $f([t]) = f([t'])$. Thus, H^0 captures invariants preserved under computation (such as type preservation or conserved weights of terms).
- $H^1(\mathcal{O}_x; G)$: Measures *normalization obstructions*. An element in H^1 is represented by a 1-cocycle (an assignment of weights to rewrites that sums to zero on closed loops) modulo those coming from constructor valuations. Non-trivial H^1 classes indicate non-confluent rewrites, cycles of reductions (non-termination), or irreversible compilation steps.
- $H^2(\mathcal{O}_x; G)$: Measures *coherence obstructions*. It captures the obstructions to filling confluence diagrams with higher-dimensional conversion cells. A non-trivial class in H^2 represents a critical pair (or confluence loop) that cannot be closed by conversions or equivalence cells, pointing to deep semantic mismatches or failures of univalence/coherence.

6.2 Examples across the Spines

Example 6.4 (Spines I/II: Strict & Homotopical).

- **Henk** ($\mathcal{O}_\Pi$): The core type theory.

$$C_0 = \{\mathbf{Var}, \mathbf{Universe}, \mathbf{Pi}, \mathbf{Lam}, \mathbf{App}\}$$

The 1-cells are generated by β -reduction:

$$r_\beta : \mathbf{App}(\mathbf{Lam}(x, A, t), u) \rightarrow t[u/x]$$

$\partial_1(r_\beta) = [t[u/x]] - (\mathbf{App} + \mathbf{Lam} + [t] + [u])$. The 2-cells consist of η -conversions:

$$e_\eta : \mathbf{Lam}(x, A, \mathbf{App}(t, x)) \equiv_\eta t$$

Since Henk is strongly normalizing and confluent (Church-Rosser), the higher cohomology $H^k(\mathcal{O}_\Pi; \mathbb{Z})$ for $k \geq 1$ is trivial, reflecting a contractible syntactic space.

- **Anders** ($\mathcal{O}_{\text{Anders}}$): Adds univalence and Glue types. The univalence axiom represents a path-space equivalence between isomorphic types. This introduces higher-dimensional coherences (Glue cells) acting as 3-cells, preventing non-trivial 2-dimensional obstructions.
- **Urs** ($\mathcal{O}_{\text{Urs}}$): Adds modalities ($\flat, \sharp$) and differential structures (forms, connections). The syntactic CW-complex of $\mathcal{O}_{\text{Urs}}$ is enriched with topological and differential cells, yielding non-trivial H^k corresponding to de Rham cohomology and Chern-Weil invariants of smooth/differential type structures.

Example 6.5 (Spine III: Category Theory).

- **Dan** ($\mathcal{O}_{\text{Dan}}$): Operational algebra. Here, term syntax encodes algebraic operations (e.g., face maps ∂_i , degeneracies s_i). The cell complex $C_*(\mathcal{O}_{\text{Dan}})$ is isomorphic to the standard simplicial chain complex of the underlying category/algebraic presentation.
- **Mike** ($\mathcal{O}_{\text{Mike}}$): Directed category theory. 1-cells are non-reversible functors/morphisms, meaning paths are directed. The boundary operator ∂_1 tracks directed change. Quadraticity constraints act as 2-cells that model the non-commutative relation profiles of monoidal/directed types.
- **Ulrik** ($\mathcal{O}_{\text{Ulrik}}$): Simplicial ∞ -categories. Generalizes Mike's 1-categories to directed simplicial spaces. The higher cells (C_k for $k \geq 2$) represent the Segal completeness conditions and Rezk completeness conditions, ensuring that the directed paths compose coherently up to higher homotopies.

7 Spectral Sequence of Language Categories

Unlike a simple linear filtration, the language category $\mathcal{L}$ is organized as a *lattice* (a partially ordered set $\mathcal{I}$) with multiple join-paths. To compute the cohomology of the total language $\mathcal{L}$ from its sublanguages, we can employ the Bousfield-Kan spectral sequence for the homotopy colimit over the poset category $\mathcal{I}$:

$$E_2^{p,q} = H^p(\mathcal{I}; \mathcal{H}^q) \implies H^{p+q}(\mathcal{L})$$

where $\mathcal{H}^q : \mathcal{I} \rightarrow \mathbf{Ab}$ is the functor mapping each language $\mathcal{O}$ to its cohomology $H^q(\mathcal{O})$, and $H^p(\mathcal{I}; -)$ denotes the cohomology of the category $\mathcal{I}$.

Alternatively, one can impose an integer rank filtration on the syntactic complex $C^*(\mathcal{L})$ by defining the depth p of a language in the lattice. Let $F_p C^*(\mathcal{L})$ be the subcomplex generated by all languages of depth $\leq p$. The associated spectral sequence is:

$$E_1^{p,q} = H^{p+q}(F_p/F_{p-1}) \implies H^{p+q}(\mathcal{L})$$

where the relative complex F_p/F_{p-1} isolates the “new” syntactic primitives introduced exactly at depth p .

p	Language	Branch	Primitives added	$ C_0 $
0	Henk	CTT	Π, U^n	5
1	Frank	CTT	+ Ind	8
2a	Christine	CTT	+ Σ, Id	14
2b	Per	CTT	+ $\Sigma, \text{Id}, \text{W}, 0, 1, 2$	26
3	Anders	CTT	+ Path, I, Glue, HComp, HIT	40+
4a	Felix	CTT/Modal	+ $\flat, \sharp, \Im, \otimes, \text{Plot}$	10
4b	Jack	CTT/Modal	+ Spectra, Susp, Grpd, Forms, DiffKU	12
4c	Urs	CTT/Modal	+ Qubit, Linear, Config, Braid	7
—	Dan	Algebra	Groups, Simplices, Chains, Boundaries	20
—	Mike	Directed	Hom, $\otimes, \int, \coprod, \multimap$	13
—	Ulrik	Directed	+ $\mathbb{I}^\rightarrow, \text{Ext}, \text{Tw}, \leq$	25

Table 2: The language lattice: filtration indices and constructor counts. Note the split at $p = 2$ (Christine/Per), the four sublanguages of Urs at $p = 4$ (Homotopy, SupSmth, DiffK, Urs), and the separate branch indices for Dan, Mike, and Ulrik.

Definition 7.1 (Morphisms in $\mathcal{L}$). Morphisms $f : \mathcal{O}_x \rightarrow \mathcal{O}_y$ are classified as:

1. $\mathcal{F}_{\text{enrich}}$: Enrichment (type inference, elaboration).
2. $\mathcal{F}_{\text{simp}}$: Simplification (type erasure, extraction).
3. $\mathcal{F}_{\text{trans}}$: Translation (compilation across branches).
4. $\mathcal{F}_{\text{eval}}$: Evaluation (normalization to values).
5. $\mathcal{F}_{\text{norm}}$: Normalization ($\beta\eta$ -reduction).
6. $\mathcal{F}_{\text{certify}}$: Certification (proof checking).

Cross-branch morphisms (e.g. **Lift**: Dan $\rightarrow$ Mike, **Core**: Ulrik $\rightarrow$ Anders) are structure-changing and do not preserve the term type.

7.1 Functorial Conversions and Cross-Branch Morphisms

The language category $\mathcal{L}$ is not merely a set of objects; the relationships between the branches are mediated by structured functorial conversions. The primary conversions, their statuses, and associated information losses are detailed in Table 3.

Functor / Conversion	Source	Target	Status	Semantics & Information Loss
Lift	Dan	Mike	Yes	Semantical lifting of presentation generators and equations to 1-categorical structures.
Forget	Mike	Dan	Partial	Truncates categories back to operation-based presentation generators.
Free	Mike	Ulrik	Yes	Fully faithful embedding of directed 1-categories into directed ∞ -categories.
Trunc	Ulrik	Mike	No	Cannot be done functorially without losing higher-dimensional coherence.
Core	Ulrik	Anders	Yes	Groupoid Core: discards directed path structure, retaining only the sub-groupoid of isomorphisms.
Disc	Anders	Ulrik	Yes	Discrete embedding: interprets ∞ -groupoids as discrete ∞ -categories via disc modality.
Forget	Anders	Mike	Partial	1-category approximation: projects higher paths onto 1-category hom-sets.
Groupoid	Mike	Anders	Yes	Forgetful functor from directed 1-categories to spaces of isomorphism paths.
Lift (Composite)	Dan	Ulrik	Yes	Composition of Lift and Free : maps operations to Rezk ∞ -categories.
Forget	Ulrik	Dan	No	Directed simplicial spaces are too rich to map back to discrete operations directly.
Forget	Anders	Dan	No	Undirected spaces cannot be functorially mapped to discrete presentation equations.
Modal	Anders	Felix	Yes	Extends cubical HoTT with cohesive modalities ($\flat$, $\sharp$, $\Im$) and super-grades.
Diff	Felix	Jack	Yes	Stable extension to spectra, suspensions, forms, and differential K-theory.
Urs	Jack	Urs	Yes	Extends differential K-theory with configuration spaces, braids, and linear types.

Table 3: Primary conversions and functors between the language layers.

7.1.1 Deep Dive: The Lift Functor ($\mathbf{Dan} \rightarrow \mathbf{Ulrik}$)

The **Lift** functor acts as a semantical compilation bridge that transports combinatorial presentations from the operational layer **Dan** to the directed synthetic type theories of **Mike** and **Ulrik**. Rather than a simple syntax translator, it elaborates discrete presentations into rich categorical models in four distinct phases:

1. **Signature Ingestion:** Reads the generator set, face maps (∂_i) , degeneracy maps, and algebraic presentation relations directly from the **Dan** syntax.
2. **Semantical Lifting:** Maps the ingested signature into category-theoretic structures, translating generator equations into functors, categories, and sheaves.
3. **Elaboration to STT:** Translates composition equations to identity paths ($\mathbf{EId}$) in **Ulrik**. The directed interval $\mathbb{I}^\rightarrow$ is employed to model the domains and boundary conditions.
4. **Metatheory Absorption:** Automatically equips the lifted object with the abstract category-theoretic library of **Ulrik** (including Yoneda lemmas, limits, adjunctions, and Kan extensions).

7.1.2 Deep Dive: The Groupoid Core ($\mathbf{Ulrik} \rightarrow \mathbf{Anders}$)

The translation from **Ulrik** to **Anders** is a structural forgetful conversion that extracts the underlying homotopy type (the ∞ -groupoid of isomorphisms) from a directed ∞ -category. Since the simplicial directed interval $\mathbb{I}^\rightarrow$ and directed hom-types $(x \rightarrow y)$ of **Ulrik** do not exist in the cubical undirected world of **Anders**, the functor operates as follows:

- Discards directed arrows, keeping only the sub-groupoid A^{core} containing morphisms with valid inverses.
- Discards the Segal and Rezk completeness conditions, transforming directed category composition into symmetric path composition.
- Replaces the directed interval $\mathbb{I}^\rightarrow$ with the cubical symmetric interval I (with connection and reversal operations).

7.1.3 Case Study: Möbius Strip Representation

To illustrate the conversions, Table 4 compares the representation of a Möbius Strip across the three primary layers (**Dan**, **Ulrik**, and **Anders**).

Aspect	Dan (Operational)	Ulrik (Simplicial STT)	Anders (Cubical HoTT)
Framework	Computer algebra presentations	Simplicial Type Theory with $\mathbb{I}^{\rightarrow}$	Cubical HoTT with modal operators
Mathematical Type	Combinatorial simplex complex with boundary relations	Rezk ∞ -category built from simplicial diagrams	Univalent fiber bundle: $\sum_{x:S^1} \text{Moebius}(x)$
Twist Encoding	Equations on face maps (∂_i)	Directed paths over $\mathbb{I}^{\rightarrow}$ satisfying Segal conditions	Univalent path transport (ua) using boolean negation ($\text{true} \equiv \text{false}$)
Homotopy Type	Contractible local complex (*)	∞ -category whose core homotopy type is S^1	Homotopy equivalence to the circle S^1
Abstraction Level	Combinatorial building blocks	Global synthetic ∞ -category	Global synthetic univalent bundle
Functor Action	Ingested by Lift	Lifted to directed ∞ -category	Extracted via Core as undirected ∞ -groupoid
Information Kept	Full combinatorial generators	Full directed structure	Homotopy skeleton; directed composition is lost

Table 4: Comparison of the Möbius Strip representation across the layers.